Research on a Graph Neural Network and Transformer Fusion Model for Network Intrusion Detection

1st Jiangmin Tang *
Guangdong University of Science and Technology
Dongguan, China
*2447037887@qq.com

2026 7th International Conference on Big Data & Artificial Intelligence & Software Engineering (ICBASE) | 979-8-3195-4190-1/26/$31.00 ©2026 IEEE | DOI: 10.1109/ICBASE70763.2026.11619522

***Abstract*—With the continuous evolution of cyberattacks, traditional network intrusion detection methods often suffer from limited feature representation, insufficient modeling of attack correlations, and weak detection of minority-class attacks. To address these issues, this paper proposes GNN-Transformer-IDS, a fusion model that integrates graph neural networks and Transformer for network intrusion detection. The model first preprocesses network traffic data and constructs a traffic graph based on sample similarity. A graph neural network is then used to learn local structural correlations among traffic samples, while a Transformer encoder captures global feature dependencies through multi-head self-attention. The extracted local and global features are fused and fed into a classifier for intrusion detection. Experiments are conducted on the UNSW-NB15 and CICIDS2017 datasets. The proposed model achieves Accuracy, Precision, Recall, and F1-score of 95.83%, 95.27%, 96.18%, and 95.72% on UNSW-NB15, and 98.46%, 98.21%, 98.73%, and 98.47% on CICIDS2017. Compared with GCN, GAT, and Transformer models, GNN-Transformer-IDS improves detection performance and reduces false alarms, demonstrating its effectiveness in complex attack scenarios.**

Keywords*—*Network intrusion detection; graph neural network; Transformer; traffic graph construction; attention mechanism; deep learning

I. Introduction

With the rapid development of cloud computing, the Internet of Things, mobile Internet, and big data, cyberspace data interactions continue to expand, and network security threats have become increasingly complex. Attacks such as DDoS, port scanning, malware, brute-force attacks, and web attacks pose serious challenges to system stability and data security. As a key component of network security defense, intrusion detection can identify abnormal behaviors and potential attacks by analyzing network traffic features [1].

Traditional intrusion detection methods mainly include rule-based and machine learning-based approaches. Rule-based methods rely on manually designed attack signatures and are effective for known attacks, but they are difficult to adapt to unknown or variant attacks. Machine learning methods improve detection flexibility by learning classification boundaries from data, but their performance often depends on manual feature selection and remains limited when processing complex high-dimensional traffic [2]. In recent years, deep learning methods have been widely used in intrusion detection. CNNs can extract local features, RNNs can model sequential relationships, and Transformers can capture global dependencies through self-attention. However, a single deep learning model usually focuses on one type of feature representation and struggles to model both structural associations among traffic samples and global contextual dependencies [3].

In real network environments, traffic samples are not completely independent. Samples belonging to the same attack type often share similarities in connection behavior, protocol features, packet statistics, and communication patterns. Therefore, modeling traffic samples as a graph structure can help capture potential relationships among them. Graph neural networks can learn local structural features through information propagation between nodes, providing an effective method for representing complex relationships among network entities and traffic flows [4]. However, their ability to model long-range global dependencies remains limited. Transformers, by contrast, are effective in capturing global contextual relationships through multi-head self-attention. Therefore, combining graph neural networks with Transformers can integrate the advantages of local structural modeling and global dependency modeling [5].

Based on this motivation, this paper proposes GNN-Transformer-IDS, a network intrusion detection model that integrates graph neural networks and a Transformer. The model first preprocesses network traffic samples and constructs a traffic graph based on sample similarity. It then uses a graph neural network to aggregate neighborhood information and extract structural correlation features. The resulting node representations are further input into a Transformer encoder to model global dependencies. Finally, the structural and global features are fused, and intrusion detection results are generated through a fully connected classifier. To alleviate class imbalance, a class-weighted cross-entropy loss function is introduced to improve the detection performance for minority-class attacks.

The main contribution of this paper is the design of a fusion model that simultaneously captures structural associations and global dependencies in network traffic. By constructing a traffic graph using K-nearest-neighbor relationships, the model mines potential correlations among traffic samples. Experiments are conducted on the UNSW-NB15 and CICIDS2017 datasets. Compared with SVM, Random Forest, CNN, LSTM, CNN-LSTM, GCN, GAT, and Transformer, the proposed model achieves better accuracy, recall, F1-score, and false alarm rate, demonstrating its effectiveness in network intrusion detection.

II. Intrusion Detection Model Based on the Fusion of Graph Neural Networks and Transformer

A. Overall Model Architecture

To capture both structural correlations among network traffic samples and global dependencies, this paper proposes GNN-Transformer-IDS, a fusion model for network intrusion detection. The model first preprocesses raw traffic data into numerical features and constructs a traffic graph based on sample similarity, where each traffic record is treated as a node and edges represent feature similarity. Then, a graph neural network extracts local structural features, while a Transformer encoder models global dependencies among traffic features. Finally, the fused features are fed into a classifier to identify normal and attack traffic, as shown in Figure 1.

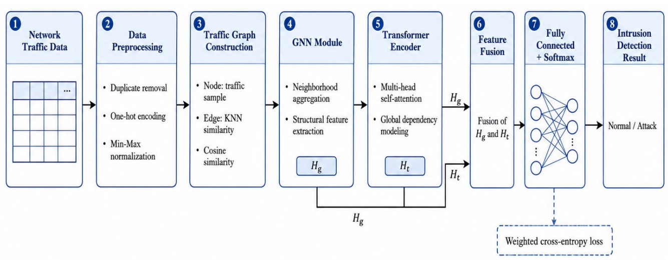

Figure 1. Overall Architecture of the GNN-Transformer-IDS Model

B. Traffic Graph Construction

To describe the potential correlations among network traffic samples, this paper constructs the traffic data as a graph structure:

$$\mathrm{G} = (\mathrm{V}, \mathrm{E}) \tag{1}$$

where V and E denote the node and edge sets, respectively. Each node represents a traffic sample, and edges indicate similarity relationships between samples. Cosine similarity is used to measure sample correlations:

$$s_{ij} = \frac{\mathrm{x}_i \cdot \mathrm{x}_j}{\|\mathrm{x}_i\| \|\mathrm{x}_j\|} \tag{2}$$

For each node, the K nodes with the highest similarity are selected as its neighboring nodes, thereby constructing the adjacency matrix:

$$A_{ij} = \begin{cases} 1, & \mathrm{x}_j \in \mathrm{N}_K(\mathrm{x}_i) \\ 0, & otherwise \end{cases} \tag{3}$$

where $\mathrm{N}_K(\mathrm{x}_i)$ denotes the K nearest neighboring nodes of sample x_i. In this way, the model can use the graph structure to represent the local similarity relationships among network traffic samples.

C. Feature Extraction Based on Graph Neural Networks

After constructing the traffic graph, this paper adopts a graph neural network to aggregate and update node features. The node update process of the l-th layer graph neural network can be expressed as:

$$\mathrm{H}^{(l+1)} = \sigma(\hat{\mathrm{A}}\mathrm{H}^{(l)}\mathrm{W}^{(l)}) \tag{4}$$

where $\mathrm{H}^{(l)}$ denotes the node feature representation of the l-th layer, $\mathrm{W}^{(l)}$ is a learnable weight matrix, $\sigma(\cdot)$ is a nonlinear activation function, and $\hat{\mathrm{A}}$ denotes the normalized adjacency matrix. The initial input is:

$$\mathrm{H}^{(0)} = \mathrm{X} \tag{5}$$

After propagation through multiple graph neural network layers, the model can obtain node representations containing neighborhood structural information:

$$\mathrm{H}_g = \mathrm{GNN}(\mathrm{X}, \mathrm{A}) \tag{6}$$

where H_g denotes the structural correlation features extracted by the graph neural network.

D. Global Feature Modeling Based on Transformer

Graph neural networks mainly capture local neighborhood structures. To model global dependencies, this paper feeds the node representations generated by the GNN into a Transformer encoder, where the self-attention mechanism is expressed as follows:

$$\mathrm{Attention}(\mathrm{Q}, \mathrm{K}, \mathrm{V}) = \mathrm{softmax}\left(\frac{\mathrm{QK}^T}{\sqrt{d_k}}\right)\mathrm{V} \tag{7}$$

where Q, K, and V denote the query matrix, key matrix, and value matrix, respectively, and d_k denotes the dimension of the key vector. The multi-head attention mechanism can be expressed as:

$$\mathrm{MultiHead}(\mathrm{Q}, \mathrm{K}, \mathrm{V}) = \mathrm{Concat}(\mathrm{head}_1, \cdots, \mathrm{head}_h)\mathrm{W}^O \tag{8}$$

$$\mathrm{head}_i = \mathrm{Attention}(\mathrm{QW}_i^Q, \mathrm{KW}_i^K, \mathrm{VW}_i^V) \tag{9}$$

where h denotes the number of attention heads, and W_i^Q, W_i^K, W_i^V, and W^O are learnable parameter matrices. After Transformer encoding, the global dependency features can be obtained as:

$$\mathrm{H}_t = \mathrm{Transformer}(\mathrm{H}_g) \tag{10}$$

E. Feature Fusion and Classification

To simultaneously utilize the structural features extracted by the graph neural network and the global features modeled by Transformer, this paper adopts a fusion module to generate the final feature representation:

$$\mathrm{H}_f = \alpha\mathrm{H}_g + (1 - \alpha)\mathrm{H}_t \tag{11}$$

where H_f denotes the fused feature representation, and α is the feature fusion weight. Then, the fused features are fed into a fully connected classifier:

$$\hat{\mathrm{y}} = \mathrm{softmax}(\mathrm{W}_c\mathrm{H}_f + \mathrm{b}_c) \tag{12}$$

where W_c and b_c denote the weight and bias of the classification layer, respectively, and $\hat{\mathrm{y}}$ denotes the prediction result of the model.

The model is trained using the cross-entropy loss function:

$$L = -\sum_{i=1}^{N} y_i \log(\hat{y}_i) \tag{13}$$

To address the class imbalance problem in intrusion detection data, this paper further introduces class weights, resulting in the weighted cross-entropy loss function:

$$L = -\sum_{i=1}^{N} w_i\, y_i log(\hat{y}_i) \tag{14}$$

where w_i denotes the class weight corresponding to the i-th class sample. Through this loss function, the model can pay more attention to minority-class attack samples during training.

F. Model Workflow

In summary, the overall computational process of the proposed model can be expressed as:

$$G = (V, E) = Graph(X) \tag{15}$$

$$H_g = GNN(X, A) \tag{16}$$

$$H_t = Transformer(H_g) \tag{17}$$

$$H_f = Fusion(H_g, H_t) \tag{18}$$

$$\hat{y} = Classifier(H_f) \tag{19}$$

The proposed model integrates GNN-based structural learning and Transformer-based global dependency modeling to improve intrusion detection in complex network scenarios.

III. Experiments and Result Analysis

A. Experimental Settings

To evaluate the proposed GNN-Transformer-IDS model, comparative experiments, ablation studies, and per-class analyses were conducted on the UNSW-NB15 and CICIDS2017 datasets. The experiments were performed on Ubuntu 20.04 with an Intel Xeon Silver 4210R CPU, an NVIDIA RTX 3090 GPU, and 64 GB RAM, and the model was implemented using PyTorch 2.0 and PyTorch Geometric. During training, Adam was used with a learning rate of 0.001, a weight decay of 5×10^{-4}, a batch size of 128, a maximum of 100 epochs, and a dropout rate of 0.3. The datasets were split into training, validation, and test sets at a ratio of 7:1:2. Duplicate samples, abnormal fields, and records with many missing values were removed; discrete features were encoded by One-Hot encoding, and continuous features were normalized by Min-Max normalization. A class-weighted cross-entropy loss was used to reduce the influence of class imbalance. Accuracy, Precision, Recall, F1-score, and False Alarm Rate were used as evaluation metrics. SVM, Random Forest, CNN, LSTM, CNN-LSTM, GCN, GAT, and Transformer were selected as baseline models. All experiments were repeated five times with different random seeds, and the results are reported as mean ± standard deviation.

B. Overall Performance Comparison

Table 1 and Table 2 compare the performance of different models on UNSW-NB15 and CICIDS2017. All experiments were repeated five times with different random seeds, and the results are reported as mean ± standard deviation. The p-value was calculated by a two-tailed paired t-test on the F1-scores between each baseline and the proposed model. As shown in Tables 1 and 2, the proposed GNN-Transformer-IDS achieves the best Accuracy, Precision, Recall, F1-score, and FPR on both datasets, demonstrating the effectiveness of combining graph structure modeling with global dependency learning.

Table 1. Comparison of Detection Performance of Different Models on the UNSW-NB15 Dataset over Five Runs

Model	Accuracy /%	Precision /%	Recall/ %	F1/ %	FPR/ %	p
SVM	88.64 ± 0.21	87.92 ± 0.24	88.31 ± 0.22	88.11 ± 0.23	8.72 ± 0.15	<0.001
Random Forest	90.76± 0.19	90.25± 0.20	90.81 ± 0.18	90.53 ± 0.19	7.18 ± 0.13	<0.001
CNN	92.13 ± 0.17	91.68 ± 0.18	92.35 ± 0.16	92.01 ± 0.17	6.21 ± 0.12	<0.001
LSTM	92.74 ± 0.16	92.18 ± 0.17	93.06 ± 0.15	92.62 ± 0.16	5.84 ± 0.11	<0.001
CNN-LSTM	93.51 ± 0.15	92.96 ± 0.16	93.87 ± 0.14	93.41 ± 0.15	5.36 ± 0.10	<0.001
GCN	94.26 ± 0.14	93.71 ± 0.15	94.58 ± 0.13	94.14 ± 0.14	4.73 ± 0.10	0.0006
GAT	94.72 ± 0.13	94.16 ± 0.14	95.03 ± 0.12	94.59 ± 0.13	4.31 ± 0.09	0.0013
Transformer	94.95 ± 0.13	94.52 ± 0.13	95.28 ± 0.12	94.90 ± 0.12	4.18 ± 0.09	0.0021
Proposed Model	95.83 ± 0.10	95.27 ± 0.11	96.18 ± 0.09	95.72 ± 0.09	3.46 ± 0.07	Reference

Table 2. Comparison of Detection Performance of Different Models on the CICIDS2017 Dataset over Five Runs

Model	Accuracy /%	Precision /%	Recall/ %	F1/ %	FPR/ %	p-
SVM	93.25 ± 0.16	92.78 ± 0.17	93.41 ± 0.15	93.09 ± 0.16	5.47 ± 0.12	<0.001
Random Forest	95.18 ± 0.14	94.83 ± 0.15	95.36 ± 0.13	95.09 ± 0.14	4.21 ± 0.10	<0.001
CNN	96.47 ± 0.12	96.15 ± 0.13	96.62 ± 0.11	96.38 ± 0.12	3.39 ± 0.09	<0.001
LSTM	96.73 ± 0.11	96.42 ± 0.12	96.91 ± 0.10	96.66 ± 0.11	3.18 ± 0.08	<0.001
CNN-LSTM	97.21 ± 0.10	96.95 ± 0.11	97.43 ± 0.09	97.19 ± 0.10	2.76 ± 0.07	0.0008
GCN	97.64 ± 0.09	97.36 ± 0.10	97.82 ± 0.08	97.59 ± 0.09	2.43 ± 0.06	0.0016
GAT	97.92 ± 0.08	97.65 ± 0.09	98.14 ± 0.07	97.89 ± 0.08	2.18 ± 0.06	0.0029
Transformer	98.03 ± 0.08	97.78 ± 0.08	98.26 ± 0.07	98.02 ± 0.07	2.05 ± 0.05	0.0046
Proposed Model	98.46 ± 0.06	98.21 ± 0.07	98.73 ± 0.05	98.47 ± 0.05	1.62 ± 0.04	Reference

As shown in Tables 1 and 2, the proposed model achieves F1-scores of 95.72% and 98.47% on UNSW-NB15 and CICIDS2017, respectively. Compared with Transformer, the strongest baseline, the F1-score is improved by 0.82 percentage points on UNSW-NB15 and 0.45 percentage points on CICIDS2017. The corresponding p-values are 0.0021 and 0.0046, both lower than 0.05, indicating that the improvements are statistically significant. In addition, the proposed model obtains the lowest FPR on both datasets, showing that the fusion of graph structural features and global attention features improves detection performance while reducing false alarms.

C. *Parameter Sensitivity Analysis*

Since the traffic graph is constructed using K-nearest neighbor relationships, the value of K and the similarity metric directly affect graph quality. To provide a basis for parameter selection, a compact sensitivity analysis was conducted on the UNSW-NB15 dataset. The average F1-score of Backdoor, Shellcode, and Worms was used as the minority-class F1-score.

Table 3. Parameter Sensitivity Analysis on the UNSW-NB15 Dataset

Setting	F1/%	Minority F1/%
K = 5	94.82	88.41
K = 10	95.18	89.16
K = 15	95.49	89.71
K = 20	95.72	90.08
K = 25	95.64	89.96
K = 30	95.42	89.62
K = 40	95.05	88.97
K = 50	94.67	88.31
Cosine similarity	95.72	90.08
Euclidean distance	95.26	89.36
Manhattan distance	95.09	89.06
RBF kernel similarity	95.45	89.67

As shown in Table 3, the model achieves the best overall F1-score and minority-class F1-score when K = 20. When K is too small, the graph contains insufficient neighborhood information. When K is too large, weakly related samples may introduce noisy edges, which is especially harmful to minority classes. In addition, cosine similarity outperforms Euclidean distance, Manhattan distance, and RBF kernel similarity. This indicates that cosine similarity is more suitable for constructing traffic graphs because it focuses on the directional relationship between normalized feature vectors rather than absolute numerical distance. Therefore, K = 20 and cosine similarity were adopted in the final model.

D. *Some Common Mistakes*

To verify the contribution of each module, ablation experiments were conducted on both datasets. As shown in Figure 2, removing either the GNN module, Transformer module, or attention fusion mechanism leads to performance degradation. The complete GNN-Transformer-IDS model achieves the highest F1-score, confirming that structural correlation learning, global dependency modeling, and effective feature fusion all contribute to the final detection performance.

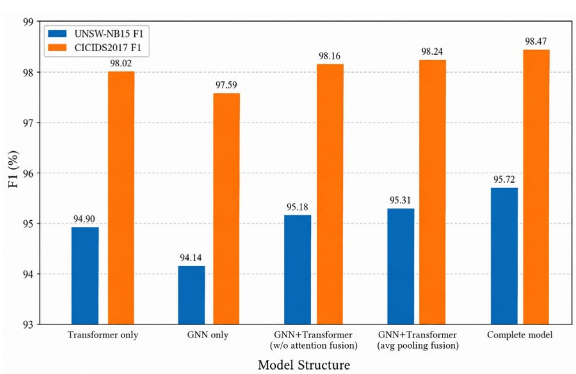

Figure 2. Comparison of Ablation Experiment Results

E. *Analysis of Classification Detection Results*

To further analyze the recognition capability of the proposed model for different attack categories, the per-class detection results on the UNSW-NB15 dataset are reported in Table 4. The model achieves relatively higher F1-scores for majority categories such as Normal, Generic, Exploits, Fuzzers, and DoS, indicating that sufficient training samples help the model learn stable traffic patterns. However, the performance for minority categories remains limited. Backdoor, Shellcode, and Worms obtain F1-scores of 90.80%, 90.31%, and 89.13%, respectively, with Worms showing the lowest result. This suggests that low-frequency attacks are still difficult to distinguish due to limited samples and possible feature overlap with other categories. Therefore, although the proposed model improves overall detection performance, minority-class detection remains a limitation that requires further improvement.

Table 4. Classification Detection Results of the Proposed Model on the UNSW-NB15 Dataset

Category	Precision/%	Recall/%	F1/%
Normal	96.83	97.21	97.02
Generic	97.42	97.86	97.64
Exploits	95.31	95.78	95.54
Fuzzers	94.72	95.06	94.89
DoS	94.18	94.63	94.40
Reconnaissance	93.84	94.27	94.05
Analysis	91.76	92.31	92.03
Backdoor	90.42	91.18	90.80
Shellcode	89.95	90.67	90.31
Worms	88.74	89.52	89.13

IV. CONCLUSION

This paper proposes GNN-Transformer-IDS for network intrusion detection. The model constructs a traffic graph using K-nearest neighbor relationships based on sample similarity. A graph neural network extracts local structural features, while a Transformer encoder captures global dependencies. Experiments on UNSW-NB15 and CICIDS2017 show that the proposed model outperforms traditional machine learning, deep learning, graph-based, and Transformer baselines. It achieves F1-scores of 95.72% and 98.47% on the two datasets, respectively, while reducing the false alarm rate. Ablation results confirm the effectiveness of the GNN module, Transformer encoder, and feature fusion mechanism. However, minority attacks such as Worms, Shellcode, and Backdoor still show lower performance. Future work will focus on graph optimization, minority-class detection, and real-time deployment.

REFERENCES

[1] Chathoth A K, Parashar K, Peng A, et al. PCAP-Backdoor: Backdoor Generator in Network Traffic for Intrusion Detection Systems[J]. ACM Transactions on Cyber-Physical Systems, 2026, 10(3): 1-25.

[2] Kwubeghari A, Ezeji N G, Okoye F A. Zero-Day Hunter: A Multi-Layered Machine Learning Framework for Real-Time Detection and Mitigation of Zero-Day Cyber Attacks[J]. ABUAD Journal of Engineering Research and Development (AJERD), 2026, 9(1): 229-237.

[3] Ali M L, Thakur K, Schmeelk S, et al. Deep learning vs. machine learning for intrusion detection in computer networks: A comparative study[J]. Applied Sciences, 2025, 15(4): 1903.

[4] Bilot T, El Madhoun N, Al Agha K, et al. Graph neural networks for intrusion detection: A survey[J]. IEEe Access, 2023, 11: 49114-49139.

[5] Govindarajan V, Muzamal J H. Advanced cloud intrusion detection framework using graph based features transformers and contrastive learning[J]. Scientific Reports, 2025, 15(1): 20511.